

重点回答

一、RadixAttention 与 RadixCache 解决的问题

多轮对话、预设的system prompt等场景会产生相同的 token 前缀。如果每次请求都重新计算这些前缀，就会浪费 Prefill 计算和显存。

RadixAttention 利用前缀的 KV 复用减少重复工作；RadixCache 用前缀树管理 token 前缀与 KV 缓存的对应关系，减少相同前缀的重复计算。新请求复用命中前缀的 KV，只计算剩余输入。请求结束后缓存可保留，空间不足时淘汰未被使用的部分。

二、Page sized KV cache 与 prefix reuse 在pipeline中的位置

Prefix reuse 发生在请求准备与调度阶段的前缀匹配中，命中的前缀免去重复计算，未命中的部分仍需计算。

Page-sized KV cache 出现在 Prefill 和 Decode 的缓存存储与显存管理环节，将 KV 按页组织和分配。

三、PagedAttention 解决的问题以及和 SGLang 的 RadixAttention/ paged KV 机制的联系与差异

PagedAttention 解决的问题：减少连续内存分配造成的显存碎片与预留浪费，使相同显存能容纳更多并发请求。

联系和差异：它与 SGLang 的 paged KV 都涉及分页存储及访问；RadixAttention 则着重自动发现和管理跨请求的前缀复用。分页与前缀复用可以配合，并非互斥方案。vLLM现在也支持自动前缀缓存。